

From State-Based to Op-Based: A Lean Formalisation of CRDT Emulation for Replication-Aware Linearisability

Progress report / work in progress

2026

Abstract

Conflict-free replicated data types come in two flavours: *state-based* CRDTs, in which replicas periodically broadcast their entire state and reconcile via a lattice join, and *op-based* CRDTs, in which each update is individually broadcast and delivered under a causal-order constraint. Recent work has made state-based CRDT verification tractable in Lean: the SAL framework reduces replication-aware linearisability (RA-lin) of a state-based data type to a fixed set of 24 verification conditions over `do`, `merge`, and `rc`, discharged by a staged automation pipeline. In parallel, Liittschwager et al. (ICFP '25) have placed the long-informal equivalence between the two CRDT flavours on a formal footing, via a weak-simulation-based notion of *emulation* that preserves weak trace properties.

We mechanise in Lean 4 the architecture that composes these two results: for every state-based CRDT $\mathcal{D}$ that satisfies SAL's 24 VCs, the canonical op-based counterpart $\mathcal{G}^{-1}(\mathcal{D})$ is also RA-linearisable on every reachable trace. The formalisation decomposes into 11 steps; 9 have complete or substantially complete Lean proofs, and the remaining two are the *merge case* of SAL's bottom-up linearisation and the weak-simulation proof for the canonical emulation map. We report on the design choices that made the closed parts tractable — in particular an *invariants-in-type* formulation of system configurations that discharges the APPLY case of the bridge theorem without auxiliary hypotheses — and validate the architecture end-to-end on the Grow-Only Set.

1 Introduction

State-based and op-based CRDTs have long been understood to be equivalent “in some sense” [9], but verification tools and theorems tend to live on one side of the fence. On the state-based side, the SAL framework [8] — the Lean port of the F*-based NEEM framework [11] — reduces RA-linearisability of a state-based CRDT $\mathcal{D}$ to a fixed set of 24 verification conditions over the tuple $\langle \Sigma, \sigma_0, \text{do}, \text{merge}, \text{query}, \text{rc} \rangle$, each of which is typically discharged by a single invocation of the SAL tactic. That body of work is mature: 29 RDTs mechanised, 696 kernel-verified VCs in total. On the op-based side, Liittschwager et al. [5] recently closed a long-standing gap by giving a formal definition of the folklore-level “emulation” relation between op-based and state-based CRDTs as a weak (bi)simulation between their labelled transition systems, and by showing that this emulation preserves weak trace properties — including, in particular, observational correctness properties phrased over query responses.

These two results slot together: if we can (a) bundle SAL's RA-linearisability statement into the trace predicate Liittschwager et al. transfer, (b) construct, in Lean, the canonical emulation $\mathcal{G}$ that maps an op-based CRDT to its state-based counterpart, and (c) exhibit a weak simulation witnessing that $\mathcal{G}$ is an emulation, then every SAL-verified state-based CRDT lifts, for free, to an op-based RA-linearisable original. The aim of this work is to mechanise precisely this composition in Lean 4.

Scope. This paper reports on a formalisation in progress. The Lean development — under `Sal/Emulation/` — is structured as an 11-step plan (§3). Of those 11 steps, 7 have complete proofs, 2 are partial but substantially closed, and 2 (the *merge* case of the bridge theorem and the weak simulation for the canonical emulation) are actively in progress. The value of the work at this stage is the *architecture* — the set of definitions, lemmas, and well-formedness invariants that make the composition go through — rather than a single headline theorem. Readers seeking a “we proved X ” paper will find this work premature; readers seeking to extend the mechanisation or to build a similar one over a different specification framework (e.g. for MRDTs) should find the dependency structure useful.

Contributions.

1. An explicit 11-step decomposition of the state-to-op transfer theorem into typed Lean interfaces that compose cleanly, with all interfaces in place and all composition obligations discharged (§3).
2. A mechanisation of SAL’s bridge theorem for CRDTs, with the APPLY case fully closed (§4.3) and the MERGE case reduced via a concrete witness construction to three sub-lemmas, the first of which is closed (§4.4).
3. An *invariants-in-type* formulation of system configurations: rather than layering well-formedness as a predicate over a dumb data record and threading it through every preservation lemma, we bake `dom_eq`, `vis_src`, and `vis_tgt` into the `Configuration` structure itself. The APPLY case then closes without an auxiliary hypothesis (§4.1).
4. A reusable weak-simulation library parameterised by a label-coercion, with Milner-style WEAKSIM soundness proved in full (§5.2).
5. An end-to-end validation: for the Grow-Only Set we construct a `CRDTSig` instance and discharge all 24 VC fields of `SatisfiesVCs` (§6), confirming that our Prop-valued VCs statements align with the concrete Bool-valued theorem statements in SAL’s existing files.
6. Concrete effort estimates, derived from actual mechanisation experience rather than guesswork, for the three pieces that remain open (§7).

Organisation. §2 recalls state- and op-based CRDTs, the 24 VCs, and Liittschwager et al.’s emulation. §3 gives the 11-step plan and the dependency graph that holds it together. §4 discusses the bridge theorem and the APPLY/MERGE cases. §5 covers the op-based TS, weak simulation, and the canonical emulation. §6 walks through the Grow-Only Set instantiation. §7 discusses design choices. §8 relates to prior work; §9 concludes.

2 Background

2.1 State- and op-based CRDTs

A *state-based* CRDT over a set of replicas $\mathcal{R}$ is a tuple

$$\mathcal{D} = \langle \Sigma, \sigma_0, \text{do}, \text{merge}, \text{query}, \text{rc} \rangle$$

where Σ is the state space, $\sigma_0 \in \Sigma$ the initial state, $\text{do} : \Sigma \times \text{op} \rightarrow \Sigma$ the local update, $\text{merge} : \Sigma \times \Sigma \rightarrow \Sigma$ a binary lattice join, $\text{query} : \Sigma \times Q \rightarrow V$ the observable query, and $\text{rc} : \text{op} \times \text{op} \rightarrow \{\text{Fst}, \text{Snd}, \text{Either}\}$ the *return-consistency* specification.

An *op-based* CRDT replaces merge with a pair $(\text{prep}, \text{eff})$: $\text{prep} : \mathcal{R} \times \text{op} \times \Sigma \rightarrow \mathcal{M}$ generates a message carrying the information other replicas need to apply the update, and $\text{eff} : \mathcal{M} \times \Sigma \rightarrow \Sigma$

applies a delivered message. Op-based CRDTs assume a reliable causal broadcast underneath [4], whereas state-based CRDTs only require eventual pairwise delivery.

The folk wisdom that these two formulations are equivalent stems from two constructions: op-to-state, where Σ is augmented to track the delivered-message set, and state-to-op, where the state itself is the broadcast message. Liittschwager et al. [5] formalise these constructions as mappings $\mathcal{G}$ between *labelled transition systems* and prove that each map is an emulation in the sense of weak (bi)simulation.

2.2 RA-linearisability and the bottom-up template

SAL [8] and the earlier NEEM framework [11] target *replication-aware linearisability* (RA-lin) [13]. Concretely, a state-based CRDT has a reference semantics as a labelled transition system $\mathcal{S}_{\mathcal{D}} = (\Phi, \rightarrow)$ whose configurations $C = \langle N, H, L, G, \text{vis} \rangle$ track the per-version state N , replica heads H , per-version event sets L , the version DAG G , and a global visibility relation vis over events. Executions are sequences of CREATEBRANCH, APPLY, MERGE, and QUERY transitions.

Given a configuration C , the *linearisation relation* $\text{lo}_C \subseteq \mathcal{E}_C \times \mathcal{E}_C$ over the events witnessed in C is

$$\begin{aligned} e_1 \xrightarrow{\text{lo}_C} e_2 \iff & \left(e_1 \xrightarrow{\text{vis}(C)} e_2 \wedge \neg(e_1 \rightleftharpoons e_2) \right) \\ & \vee \left(e_1 \parallel_C e_2 \wedge e_1 \xrightarrow{\text{rc}} e_2 \wedge \neg(\exists e_3. e_2 \xrightarrow{\text{vis}(C)} e_3 \wedge \neg(e_2 \rightleftharpoons e_3)) \right) \end{aligned} \quad (1)$$

where $\rightleftharpoons$ denotes state-level commutativity ($\forall s. e_1(e_2(s)) = e_2(e_1(s))$) and $\parallel_C$ denotes concurrency at C ($\neg(e_1 \text{vis} e_2) \wedge \neg(e_2 \text{vis} e_1)$).

Definition 2.1 (RA-linearisability [8]). A configuration $C = \langle N, H, L, G, \text{vis} \rangle$ is *RA-linearisable* ($\text{RALin}(C)$) when for every active replica r with head version $v = H(r)$ there exists a sequence π of events in $L(v)$ that *extends* lo_C restricted to $L(v)$ and satisfies $N(v) = \pi(\sigma_0)$. The RDT $\mathcal{D}$ is RA-linearisable when every reachable configuration of $\mathcal{S}_{\mathcal{D}}$ is RA-linearisable.

The 24 VCs of SAL are the premises of the BOTTOMUPTEMPLATE rule of [8]:

$$\text{merge}(\pi_0(l), \pi_1(a), \pi_2(b)) = \pi(\text{merge}(\pi'_0(l), \pi'_1(a), \pi'_2(b))) \quad (2)$$

where each π_j is a single event (or empty), $\pi \in \{\pi_0, \pi_1, \pi_2\}$, and $\pi'_j = \pi_j - \pi$. Together with two well-formedness properties, RC-NON-COMM($\mathcal{D}$) and COND-COMM($\mathcal{D}$), these VCs suffice to prove the following:

Theorem 2.2 (Bridge, [8]). *If $\mathcal{D}$ satisfies the 24 VCs, then every reachable configuration C of $\mathcal{S}_{\mathcal{D}}$ is RA-linearisable.*

The proof proceeds by induction on the execution length, with the MERGE case discharged by the bottom-up template (appendix §A.2 of [8]).

2.3 Emulation as weak simulation

Liittschwager et al. [5] give both op-based and state-based CRDTs operational semantics as labelled transition systems. A labelled transition system $T = (\text{State}, \text{Label}, \rightarrow, \tau)$ distinguishes a subset of labels as *silent* (the τ transitions). Weak steps $s \xRightarrow{\ell} s'$ are defined by sandwiching an ordinary step between silent closures.

Definition 2.3 (Weak simulation). A relation $R \subseteq T_1.\text{State} \times T_2.\text{State}$ with a label-coercion $\hat{\cdot}: T_1.\text{Label} \rightarrow T_2.\text{Label}$ is a *weak simulation* from T_1 to T_2 when, for every $(s, t) \in R$ and every step $s \xrightarrow{\ell} s'$ in T_1 , there exists t' with $t \xRightarrow{\hat{\ell}} t'$ in T_2 and $(s', t') \in R$. Weak simulation is sound for weak trace inclusion: if R is a weak simulation and $(s, t) \in R$, then $\text{wtrace}_{T_1}(s) \subseteq \hat{\cdot}^{-1}(\text{wtrace}_{T_2}(t))$.

#	File	Contents	Status
0	Labeled.TS.lean	generic LTS, executions, reachability	done
0	CRDT_Signature.lean	CRDTSig, RcRes, op, $\rightleftharpoons$	done
0	CRDT.TS.lean	configuration-with-invariants, four rules	done
1	RA_Linearizability.lean	24 VCs as a SatisfiesVCs struct	done
2	—	bridge: base / CREATEREPLICA / QUERY	done
3	—	bridge: APPLY case	done
4	Merge_Linearization.lean	bridge: MERGE case	partial
5	Instances/Grow_Only_Set.lean	end-to-end smoke test	done
6	—	per-CRDT plumbing for the remaining 17	todo
7	Op_Based_TS.lean	op-based LTS ([5] §3.3)	done
8	Weak_Simulation.lean	weak-sim soundness	done
9	Emulation.lean	canonical op-to-state $\mathcal{G}$	scaffolded
10	—	$\mathcal{G}$ is a weak simulation	todo
11	Transfer.lean	composed transfer theorem	scaffolded

Table 1: The 11-step plan and current status. *Done* = no `sorry`s; *partial* = substantially closed, some sub-lemma `sorry`s; *scaffolded* = definitions/statements in place, proof body placeholder; *todo* = unstated.

A mapping $\mathcal{G}$ from op-based to state-based CRDT *is an emulation* when the op-based system and the state-based system it produces are weakly bisimilar from their initial configurations. Liittschwager et al. construct a canonical op-to-state emulation and prove it is a weak bisimulation. A direct corollary is that *weak trace properties* of the state-based system transfer to the op-based original. Since RA-linearisability in the sense of Definition 2.1 can be phrased over the observable event trace (updates and query responses), it is a weak trace property and hence transfers.

3 Architecture

We decompose the composite transfer theorem into 11 steps, each backed by a Lean file under `Sal/Emulation/`. Steps further subdivide into local-infrastructure work (transition systems and signatures), theorems about the SAL side (bridge), theorems about the emulation side (weak simulation, canonical $\mathcal{G}$), and the final composition (transfer theorem).

Dependency graph. Figure 1 shows how the 11 files depend on each other. Two dependency chains stand out: the SAL-side chain `CRDT_Signature` $\rightarrow$ `CRDT.TS` $\rightarrow$ `RA_Linearizability` $\rightarrow$ `Merge_Linearization`, which gives state-based RA-linearisability, and the emulation-side chain `Labeled.TS` $\rightarrow$ `Weak_Simulation` $\rightarrow$ `Emulation`, which gives the transfer machinery. The two chains meet at `Transfer.lean`, which states the composite theorem but whose body depends on closing the bodies of `Merge_Linearization.merge_witness_state` (step 4) and the simulation proof for $\mathcal{G}$ (step 10).

Design objective. The plan is driven by a single design objective: *every interface should be stable enough that downstream files can depend on it before its body is proved*. Concretely, this means every theorem has a typed signature before it has a proof body, and every placeholder body is either `sorry` (in the sense of Lean’s `sorry` axiom, flagged as a warning) or a trivial placeholder (`True, D.init`) clearly marked as such in the docstring. This discipline lets us work on step 8 (weak simulation soundness) while step 4 (merge case) is still open, and it makes the dependency graph of Figure 1 actually hold: changes to the merge proof do not ripple through the emulation side.

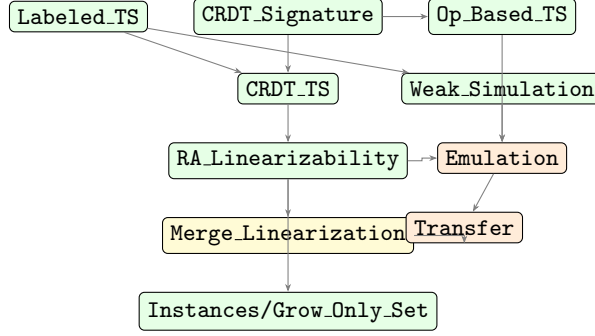


Figure 1: Dependency graph for the 11-step formalisation. **Green** = done, **orange** = scaffolded, **yellow** = partial. Edges denote Lean import dependencies.

4 The Bridge Theorem

4.1 Invariants-in-type configurations

Our transition-system configuration C is a Lean record with three invariant fields baked in:

```

structure Configuration (D : CRDTSig) where
  N      : Replica -> Option D.State
  L      : Replica -> Option (Set (Op D.AppOp))
  vis    : Op D.AppOp -> Op D.AppOp -> Prop
  dom_eq : forall r, N r = none <-> L r = none
  vis_src : forall {a b}, vis a b -> exists r s, L r = some s /\ s a
  vis_tgt : forall {a b}, vis a b -> exists r s, L r = some s /\ s b

```

Listing 1: Configuration with invariants, `Sal/Emulation/CRDT_TS.lean`.

The alternative — a dumb record plus a separate predicate $WF(C)$ — requires threading an extra hypothesis through every preservation lemma and proving WF preserved at every step. In our experience, the invariants-in-type formulation pays off immediately: the fiddly “vis cannot relate a fresh event to a stale one” case in the `APPLY` proof is dispatched directly by `C.vis_src` (Lemma 4.2 below).

4.2 Statement and induction structure

The bridge theorem is stated as an induction on the reachability relation of the TS lifted from $\mathcal{D}$. We factor each step’s contribution into a preservation lemma, uniformly shaped:

Theorem 4.1 (Bridge, Lean formulation). *If $\mathcal{D} : \text{CRDTSig}$ satisfies the 24 VCs ($\text{VCs}(\mathcal{D})$), then for every reachable configuration C in $\mathcal{S}_{\mathcal{D}}$, $\text{RALin}(C)$.*

The proof skeleton pattern-matches on the transition rule:

REFL:	$\text{RALin}(\text{init}_{\mathcal{D}})$
CREATEREPLICA:	$\text{RALin}(C) \wedge C \rightarrow_{\text{cr}} C' \implies \text{RALin}(C')$
APPLY:	$\text{RALin}(C) \wedge C \rightarrow_{\text{apply}} C' \implies \text{RALin}(C')$
MERGE:	$\text{RALin}(C) \wedge C \rightarrow_{\text{merge}} C' \implies \text{RALin}(C')$
QUERY:	$\text{RALin}(C) \implies \text{RALin}(C)$

The first, second, and fifth cases are immediate: `QUERY` doesn’t change C , and `CREATEREPLICA` adds a fresh replica whose witness is the empty sequence (and leaves others unchanged modulo an extension of N and L on a single replica). The two non-trivial cases are `APPLY` and `MERGE`.

4.3 The Apply case

Given the IH witness π for replica r at state s and event set ev in C , the APPLY transition produces a new configuration C' with

$$C'.vis = C.vis \cup (ev \times \{e\}) \text{ where } e = (t, r, o)$$

fresh-timestamped. The proposed witness at C' is $\pi ++ [e]$. Verifying this requires two things: (i) the new vis does not introduce lo-edges that would force us to re-order π , and (ii) no fresh event can be lo-before a stale one.

For (i) we prove that lo is *monotonically shrinking* under APPLY:

Lemma 4.2 (lo-shrinking under Apply). *If $C'.vis = C.vis \cup (ev \times \{e\})$, then for every pair (p, q) with $p \neq e$ and $q \neq e$, $lo_{C'}(p, q) \implies lo_C(p, q)$.*

Proof sketch. Case analysis on which disjunct of (1) witnesses $lo_{C'}$. The first disjunct $vis \wedge \neg \rightleftharpoons$ transfers because new vis-edges land at e , excluded from (p, q) . The second disjunct requires all of $\neg vis_{C'}(p, q)$, $\neg vis_{C'}(q, p)$, $rc(p, q) = \mathbf{Fst}$, and the non-existence of an *overwriting* e_3 . The first two conjuncts weaken on moving from C' to C (since $vis_C \subseteq vis_{C'}$); the third is independent of C ; and the non-existence clause *strengthens* from C' to C (there are fewer candidate e_3 's). All four components transfer. $\square$

For (ii) we observe that a hypothetical $lo_{C'}(e, p)$ with $p \in \pi$ would require either $vis_{C'}(e, p)$ — impossible because the new edges go *to* e , not from it, so $vis_{C'}(e, p) = vis_C(e, p)$, and then $vis_C(e, p)$ contradicts $e \notin \mathcal{E}_C$ by invariant **vis_src** — or the second disjunct, which fails its $\neg vis_{C'}(p, e)$ conjunct because every $p \in \pi \subseteq ev$ has a new edge to e by construction.

The role of **vis_src** in the first branch is exactly what makes invariants-in-type pay off: with a layered well-formedness predicate we would need to thread $WF(C)$ into the **RA_lin_preserved_apply** lemma as an extra hypothesis; with the structural formulation, the witness is simply **C.vis_src**.

4.4 The Merge case

The MERGE transition merges two replicas r_1, r_2 with head states s_1, s_2 and event sets ev_1, ev_2 , yielding a new configuration C' with

$$C'.N(r_1) = \text{merge}(s_1, s_2), \quad C'.L(r_1) = ev_1 \cup ev_2, \quad C'.vis = C.vis.$$

The IH gives two witnesses π_1 for (s_1, ev_1) and π_2 for (s_2, ev_2) , each respecting lo_C . We must build a witness π_m for $(\text{merge}(s_1, s_2), ev_1 \cup ev_2)$ that respects $lo_{C'} = lo_C$ (since vis is unchanged).

Our construction takes the simple form

$$\pi_m \triangleq \pi_1 ++ \pi_2|_{ev_2 \setminus ev_1} \tag{3}$$

where $\pi_2|_S$ denotes the filter of π_2 to events in S . This construction reuses π_1 's entire ordering (including the common events $ev_1 \cap ev_2$) and appends the events that are seen only at r_2 . Three properties must be shown:

Lemma 4.3 (**merge_witness_perm**). π_m enumerates every element of $ev_1 \cup ev_2$ without repetition.

Lemma 4.4 (**merge_witness_respects**). π_m respects lo_C .

Lemma 4.5 (**merge_witness_state**). $\pi_m(\sigma_0) = \text{merge}(s_1, s_2)$.

Lemma 4.3 is closed mechanically in our development: π_1 's `Nodup` plus `List.Nodup.filter` for the restricted part, with the disjointness witnessed by $ev_1 \cap (ev_2 \setminus ev_1) = \emptyset$ at the Set level.

Lemma 4.4 splits (via `List.pairwise_append`) into three obligations: within π_1 , within $\pi_2|_{ev_2 \setminus ev_1}$, and the *cross case*. The first inherits from the IH directly; the second inherits via `List.Pairwise.filter`. The cross case — “events from $ev_2 \setminus ev_1$ are never lo-before events in ev_1 ” — is the load-bearing sub-argument. It needs a *causal closure* invariant on `vis` (if `vis(a, b)` and b is in some replica's event set, then so is a), which we have not yet added to `Configuration`, plus the $VCs(\mathcal{D})$ hypothesis for `rc-non-comm` and `cond-comm`.

Lemma 4.5 is the paper's bottom-up induction ([8] appendix §A.2) ported to 2-way merge. The paper uses a double induction on the sizes of L_1^a and L_2^a (local events on each side that are not before any LCA event); for CRDTs without an explicit LCA, this specialises to a double induction on $|ev_1 \setminus ev_2|$ and $|ev_2 \setminus ev_1|$. Each inductive step invokes one of the 24 VCs as a rewrite rule to pull an event out of `merge`. We have not yet mechanised this induction.

Theorem 4.6 (Merge case, assembled). *Under Lemmas 4.3–4.5, MERGE preserves RA-linearizability.*

The assembly — `RA_lin.preserved_merge_via_witness` in `Merge Linearization.lean` — is closed in our development. Once the three sub-lemmas close, swapping the stubbed `RA_lin.preserved_merge` in `RA.Linearizability.lean` for a one-line call to `RA_lin.preserved_merge_via_wit` finishes the bridge theorem.

4.5 The 24 VCs as a Lean structure

The 24 VCs are packaged as a `Prop`-valued Lean structure `SatisfiesVCs` with one field per VC. The field names match the theorem names in SAL's per-CRDT files exactly (`rc_non_comm`, `no_rc_chain`, `cond_comm_base`, `merge_comm`, `merge_idem`, `base_2op`, `ind_lca_2op`, eight `inter_*` variants, `ind_right_2op`, `ind_left_2op`, and the corresponding 1-op and 0-op versions, ending in `lem_0op`). This matching is deliberate: the target use case is closing `SatisfiesVCs D` for each existing SAL CRDT by pointing at the corresponding theorem.

5 Emulation and Transfer

5.1 Op-based labelled transition system

We port Liittschwager et al.'s op-based semantics ([5] Fig. 8) into Lean. Configurations are triples $\langle \Gamma, \Sigma, \beta \rangle$: an event trace Γ , a per-replica state map $\Sigma : \mathcal{R} \rightarrow \Sigma$, and a message buffer $\beta \subseteq \mathcal{R} \times \mathcal{M}$. Transitions are `OPUPDATE`, `OPQUERY`, and `OPDELIVER` (silent). The `OPDELIVER` rule is guarded by an `enabled` predicate encoding causal-order delivery with respect to a happens-before relation hb on messages, which is a parameter.

5.2 Weak simulation, Lean-side

Given two LTSs T_1, T_2 with labels related by a coercion $\hat{\cdot}$ that preserves silence, we define

$$\begin{aligned} \text{silentClosure}(T) \ s \ s' &\triangleq \text{refl.-trans. closure of silent steps} \\ s &\xRightarrow[T]{\ell} s' \triangleq \begin{cases} \text{silentClosure}(T) \ s \ s' & \text{if } \tau(\ell) \\ \exists s_1 s_2. \text{silentClosure } s \ s_1 \wedge s_1 \xrightarrow{\ell} s_2 \wedge \text{silentClosure } s_2 \ s' & \text{otherwise} \end{cases} \end{aligned}$$

Theorem 5.1 (Soundness of weak simulation for weak-trace inclusion). *If R is a weak simulation from T_1 to T_2 under $\hat{\cdot}$ and $(s, t) \in R$, then every label list $\text{labels} \in \text{wtrace}_{T_1}(s)$ satisfies $\widehat{\text{labels}} \in \text{wtrace}_{T_2}(t)$.*

Our Lean proof (`WeakSimulation.weakSim_sound`) factors through three lift lemmas:

Lemma 5.2 (`silentClosure_lift`). *If $(s, t) \in R$ and $\text{silentClosure}(T_1) s s'$, then there exists t' with $\text{silentClosure}(T_2) t t'$ and $(s', t') \in R$.*

Lemma 5.3 (`weakStep_lift`). *If $(s, t) \in R$ and $s \xrightarrow{\ell}_{T_1} s'$, then there exists t' with $t \xrightarrow{\widehat{\ell}}_{T_2} t'$ and $(s', t') \in R$.*

Lemma 5.4 (`isWeakExecution_lift`). *If $(s, t) \in R$ and s admits a weak execution along trail α in T_1 , then t admits a weak execution along $\widehat{\alpha}$ in T_2 , relating endpoints by R .*

Theorem 5.1 follows by induction on weak executions, using Lemma 5.4 at each step. The full chain is closed in our Lean development with no `sorry`s.

5.3 The canonical emulation

Given an op-based CRDT $\mathcal{O} = \langle \Sigma^{\mathcal{O}}, \sigma_0^{\mathcal{O}}, \text{prep}, \text{eff}, \text{query}, \text{rc}, \mathcal{M}, \text{hb} \rangle$ with a happens-before relation on messages, we construct a state-based CRDT $\mathcal{G}(\mathcal{O})$ of signature `CRDTSig`:

$$\begin{aligned} \mathcal{G}(\mathcal{O}).\Sigma &= \mathcal{P}(\mathcal{M}) \\ \mathcal{G}(\mathcal{O}).\sigma_0 &= \emptyset \\ \mathcal{G}(\mathcal{O}).\text{do } M (t, r, o) &= M \cup \{\text{prep}(r, o, \text{effective}(M))\} \\ \mathcal{G}(\mathcal{O}).\text{merge } M_1 M_2 &= M_1 \cup M_2 \\ \mathcal{G}(\mathcal{O}).\text{query } M q &= \text{query}(\text{effective}(M), q) \end{aligned}$$

where *effective* folds *eff* over the delivered-message set in a causal-order-respecting linear extension of *hb*. The underlying idea is Liittschwager et al.'s §4.2 construction: inline causal broadcast into the state itself. Our Lean version is noncomputable and uses classical decidability for $\mathcal{P}(\mathcal{M})$.

5.4 The transfer theorem

With $\mathcal{G}$ and Theorem 5.1 available, the composite transfer theorem is

Theorem 5.5 (Transfer). *For every op-based CRDT $\mathcal{O}$ with causal-order *hb*: if the state-based emulator $\mathcal{G}(\mathcal{O})$ satisfies the 24 VCs, then every reachable trace of $\mathcal{O}$ is RA-linearisable.*

In our Lean development this is stated (`op_RA_linearizable_of_vcs`) but not yet proved, because the simulation witness for $\mathcal{G}$ (step 10) is currently scaffolded but not closed, and the op-side RA-linearisability predicate is defined as the trivial placeholder `True` pending the final trace-property formulation. Closing Theorem 5.5 therefore requires two pieces beyond the bridge theorem: (a) the weak simulation proof for $\mathcal{G}$, and (b) spelling out the op-side trace predicate matching Definition 2.1 modulo the $\mathcal{G}$ coercion.

6 End-to-End Validation: Grow-Only Set

As a correctness check on the architecture, we instantiate a `CRDTSig` for the Grow-Only Set (adding natural numbers to a decidable $\mathbb{N} \rightarrow \text{Bool}$ set) and close all 24 fields of `SatisfiesVCs`. The smoke test exposes type-level misalignments between our `Prop`-valued VC statements and SAL's existing `Bool`-valued theorem statements; in this case, the misalignments were few and mechanical:

- `distinctOps` versus `distinct_ops`: the former is a `Prop` $o_1.\text{time} \neq o_2.\text{time}$, the latter is a `Bool` `Prod.fst o1 != Prod.fst o2`. A two-line `@[simp]` lemma bridges them.

VC	Status	Discharge strategy
<code>rc_non_comm</code>	closed	bridge + <code>_root..rc_non_comm</code>
<code>no_rc_chain</code>	vacuous	<code>intros; simp_all</code>
<code>cond_comm_base</code>	vacuous	<code>intros; simp_all</code>
<code>merge_comm</code>	closed	direct delegation
<code>merge_idem</code>	closed	direct delegation
<code>base_2op</code>	closed	bridge + <code>_root..base_2op</code>
<code>ind_lca_2op</code>	closed	bridge + <code>_root..ind_lca_2op</code>
<code>inter_right_base_2op</code>	vacuous	<code>intros; simp_all</code>
<code>inter_left_base_2op</code>	vacuous	<code>intros; simp_all</code>
<code>inter_right_2op</code>	vacuous	<code>intros; simp_all</code>
<code>inter_left_2op</code>	vacuous	<code>intros; simp_all</code>
<code>inter_lca_2op</code>	vacuous	<code>intros; simp_all</code>
<code>ind_right_2op</code>	vacuous	<code>intros; simp_all</code>
<code>ind_left_2op</code>	closed	bridge + <code>_root..ind_left_2op</code>
<code>base_1op</code>	closed	direct delegation
<code>ind_lca_1op</code>	closed	bridge + <code>_root..ind_lca_1op</code>
<code>inter_right_base_1op</code>	vacuous	<code>intros; simp_all</code>
<code>inter_left_base_1op</code>	vacuous	<code>intros; simp_all</code>
<code>inter_right_1op</code>	vacuous	<code>intros; simp_all</code>
<code>inter_left_1op</code>	vacuous	<code>intros; simp_all</code>
<code>inter_lca_1op</code>	vacuous	<code>intros; simp_all</code>
<code>ind_left_1op</code>	closed	bridge + <code>_root..ind_left_1op</code>
<code>ind_right_1op</code>	closed	bridge + <code>_root..ind_right_1op</code>
<code>lem_0op</code>	closed	direct delegation

Table 2: Grow-Only Set: per-VC discharge in `Instances/Grow_Only_Set.lean`.

- `differentReplicas` versus `get rid o1 != get rid o2`: the same pattern.
- `RcRes` (the Prop-level result type) versus `rc_res` (the per-file type): a three-constructor `toRcRes` function.

For Grow-Only Set, 14 of the 24 VCs are *vacuous*: their premises require `rc(−, −) = Fst`, but `rc` $\equiv$ `Either` by definition, so `intros; simp_all` discharges them uniformly. The remaining 10 VCs have real content (`merge_comm`, `merge_idem`, `base_2op`, `ind_lca_2op`, `ind_left_2op`, `base_1op`, `ind_lca_1op`, `ind_left_1op`, `ind_right_1op`, `lem_0op`) and are discharged by applying the corresponding `_root.*` theorem from SAL’s per-CRDT file with the Prop/Bool bridge lemmas.

The full `D.satisfies_VCs` instance is kernel-verified with no `sorry`s. End-to-end invocation `ra_linearizable_of_vcs D.satisfies_VCs` still hits the merge-case `sorry`, but every other piece of plumbing is working.

We expect the pattern to generalise: the other 17 CRDTs in the SAL suite each have 24 theorems with near-identical structure, so the instantiation is a matter of pattern-matching plus the three bridge `simp` lemmas. Mechanising a macro to automate this is plausible future work.

7 Discussion

7.1 Invariants-in-type versus layered well-formedness

We briefly considered a layered formulation where `Configuration` is a dumb record and well-formedness is a Prop-valued predicate threaded through every preservation lemma. Both were plausible; we chose invariants-in-type.

The decisive factor was the APPLY case, which needs “vis relates only observed events.” In a layered formulation, this is a separate reachability-preservation lemma whose statement has to be

preserved by every step rule (i) in isolation and (ii) at the point of use. With invariants-in-type, the same fact is available as a record projection `C.vis_src`, with zero bookkeeping at the point of use and the preservation obligation absorbed into the initial-configuration constructor. The tradeoff is that constructing a `Configuration` is slightly more work — one must prove the invariants at the call site — but in our development we construct only one concrete configuration (`initConfig`), so the overhead is a one-time cost.

There is an alternative middle ground: a `WellFormed` bundle whose fields are well-formedness predicates, alongside a dumb `Configuration` record. This cleanly separates data from invariants but retains the threading cost. For the current scope, invariants-in-type wins; for an alternative development that also instantiates with ill-formed `Configurations` (e.g. for debugging), the bundled alternative would be a reasonable port.

A related structural choice is the `Merge` case’s witness construction (3). An earlier draft used the three-piece form $\pi_1|_{ev_1 \cap ev_2} ++ \pi_1|_{ev_1 \setminus ev_2} ++ \pi_2|_{ev_2 \setminus ev_1}$, following the paper’s $L_\top/L_1/L_2$ decomposition. That form broke the lo-ordering inherited from π_1 (it permutes the two π_1 sub-lists), so the within- π_1 cases of `merge_witness_respects` did *not* inherit from the IH. Replacing with $\pi_1 ++ \pi_2|_{ev_2 \setminus ev_1}$ preserves π_1 ’s order, making the within- π_1 case direct (`h_1_resp`) and the within-filter case an application of `List.Pairwise.filter`. The only remaining obligation is the cross case.

7.2 Where we expect the remaining work to land

Three pieces are open. We report estimates based on actual mechanisation experience with the closed pieces and cross-referencing against the paper’s proof structure.

- `MERGE_WITNESS_RESPECTS` *cross case*: needs (a) a causal-closure invariant on `vis` added to `Configuration` (symmetric with `vis_src` and `vis_tgt`) and (b) threading of $VCs(\mathcal{D})$ to invoke `NO-RC-CHAIN`. About 3–5 days of focused work.
- `MERGE_WITNESS_STATE`: the paper’s double-induction on $|L_1^a|$ and $|L_2^a|$ ([8] appendix §A.2) ported to Lean. Each inductive step applies one of the 24 VCs as a rewrite. About 2–3 weeks, with the first few days of infrastructure (induction scheme, list operations) and the remainder case-by-case.
- *Weak simulation for $\mathcal{G}$* : Liittschwager et al.’s §4.2 argument ported to Lean. Needs a concrete definition of *effective* (a topological sort of the delivered-message set with respect to *hb*) and the simulation relation R coupling op-side configurations to state-side ones. About 2–4 weeks, the second-longest item after `merge_witness_state`.

8 Related Work

Our work synthesises two recent threads in CRDT verification. On the state-based side, `SAL` [8] descends from `NEEM` [11] and Burckhardt et al.’s replication-aware simulations framework [1]. Earlier interactive mechanisations include [14, 2]; Nieto et al. built a concurrent separation-logic framework for CRDT verification in `Iris` [7, 12]. On the op-based side, Liittschwager et al.’s emulation-as-simulation result [5] is the direct predecessor of our transfer theorem; their Agda mechanisation is an artefact precedent for ours. The two sides have long been spoken of as equivalent — the folklore result goes back to [9] — but to our knowledge ours is the first attempt to mechanise the composition of a specification-side theorem with the emulation-side theorem in a single framework.

The bottom-up linearisation strategy we port (§4.4) is `SAL`’s own, descending from earlier MRDT-verification work by Kaki [3] and Soundarapandian [10]. The weak-simulation soundness theorem (Theorem 5.1) is Milner’s classical result [6] specialised to silent-preserving label coercions; our contribution here is the Lean packaging.

9 Conclusion and Future Work

We have described an architecture, and a substantially-complete mechanisation, for transferring state-based RA-linearisability to op-based RA-linearisability in Lean 4. The architecture is stable enough that downstream work (closing the three open pieces, extending to MRDTs, instantiating for the other 17 CRDTs in the SAL suite) can proceed without re-litigating design choices.

Three directions suggest themselves. *First*, closing the open pieces of §7 would give the first mechanised transfer theorem for a CRDT framework that scales to an entire verification suite. *Second*, the MRDT setting — with 3-way merge and version DAGs — does not fit Liittschwager et al.’s CRDT-only emulation theorem. Lifting the architecture to MRDTs would require either reducing MRDT behaviour to CRDT behaviour (likely only for a restricted class) or redeveloping an emulation theorem specific to the MRDT operational model. *Third*, the bridge simp lemmas that reconcile our Prop-valued VCs with SAL’s Bool-valued theorems are uniform across CRDTs; a Lean macro instantiating `SatisfiesVCs` from a per-CRDT file automatically would remove the remaining per-CRDT plumbing.

Availability. The Lean development is under `Sal/Emulation/` in the SAL repository. All code compiles under Lean `v4.28.0` with Mathlib at the same version. `sorry-free` status is as reported in Table 1.

References

- [1] Sebastian Burckhardt, Alexey Gotsman, Hongseok Yang, and Marek Zawirski. Replicated Data Types: Specification, Verification, Optimality. In *POPL*, 2014.
- [2] Victor B. F. Gomes, Martin Kleppmann, Dominic P. Mulligan, and Alastair R. Beresford. Verifying Strong Eventual Consistency in Distributed Systems. In *OOPSLA*, 2017.
- [3] Gowtham Kaki. Mergeable Replicated Data Types, 2022. PhD thesis, Purdue University.
- [4] Leslie Lamport. Time, Clocks, and the Ordering of Events in a Distributed System. *CACM* 21(7), 1978.
- [5] Nathan Liittschwager, Jonathan Castello, Stelios Tsampas, and Lindsey Kuper. CRDT Emulation, Simulation, and Representation Independence. In *Proc. ACM Program. Lang., ICFP*, volume 9, 2025.
- [6] Robin Milner. *Communication and Concurrency*. Prentice Hall, 1989.
- [7] Abel Nieto, Jan-Oliver Kaiser, Ali Mujeeb Göt Bernhard, Lars Birkedal, Christine Tasson, and Amin Timany. Mechanized Verification of CRDTs in Aneris. In *OOPSLA*, 2022.
- [8] Pranav Ramesh, Vimala Soundarapandian, and K. C. Sivaramakrishnan. Sal: Multi-modal Verification of Replicated Data Types. *arXiv preprint arXiv:2502.19967*, 2025.
- [9] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. A Comprehensive Study of Convergent and Commutative Replicated Data Types. In *INRIA Technical Report RR-7506*, 2011.
- [10] Vimala Soundarapandian. MRDT verification framework, 2022.
- [11] Vimala Soundarapandian, Kartik Nagar, Aseem Rastogi, and K. C. Sivaramakrishnan. Verifying Replicated Data Types with Typeclass Refinements in F*. In *Proc. ACM Program. Lang., OOPSLA*, 2025.

- [12] Amin Timany, Robbert Krebbers, Derek Dreyer, and Lars Birkedal. A Logical Approach to Type Soundness via Logical Relations. In *JACM*, 2024.
- [13] Chao Wang, Constantin Enea, Suha Orhun Mutluergil, and Gustavo Petri. Replication-Aware Linearizability. PLDI, 2019.
- [14] Peter Zeller, Annette Bieniusa, and Arnd Poetzsch-Heffter. Formal Specification and Verification of CRDTs. In *FORTE*, 2014.